

Alur Kerja Teknis Scrapebot

Tahapan, tujuan, pustaka, dan cara memilih yang terbaik

Tanggal	5 Oktober 2026
Status	Usulan teknis. Keputusan final ditetapkan setelah benchmark
Pelengkap	Rancangan Pengembangan Scrapebot Versi 2 (<code>rancangan-scrapebot-v2.pdf</code>)

Inti dokumen ini. Setiap tahap alur kerja diberi tujuan yang jelas, satu pustaka rekomendasi, dan beberapa alternatif. Semua tahap dibangun sebagai *plug-in* yang dipilih lewat berkas konfigurasi, sehingga pustaka mana pun, termasuk ScrapeGraphAI, dapat dicoba dan diganti tanpa membongkar kode. Pilihan final ditentukan oleh hasil *benchmark* pada data nyata, bukan oleh dugaan.

Daftar isi

1. Ringkasan
2. Diagram alur teknis
3. Rincian setiap tahap
4. Arsitektur plug-in agar dinamis
5. Memilih yang final melalui benchmark
6. Praktik terbaik
7. Rekomendasi awal dan langkah berikutnya

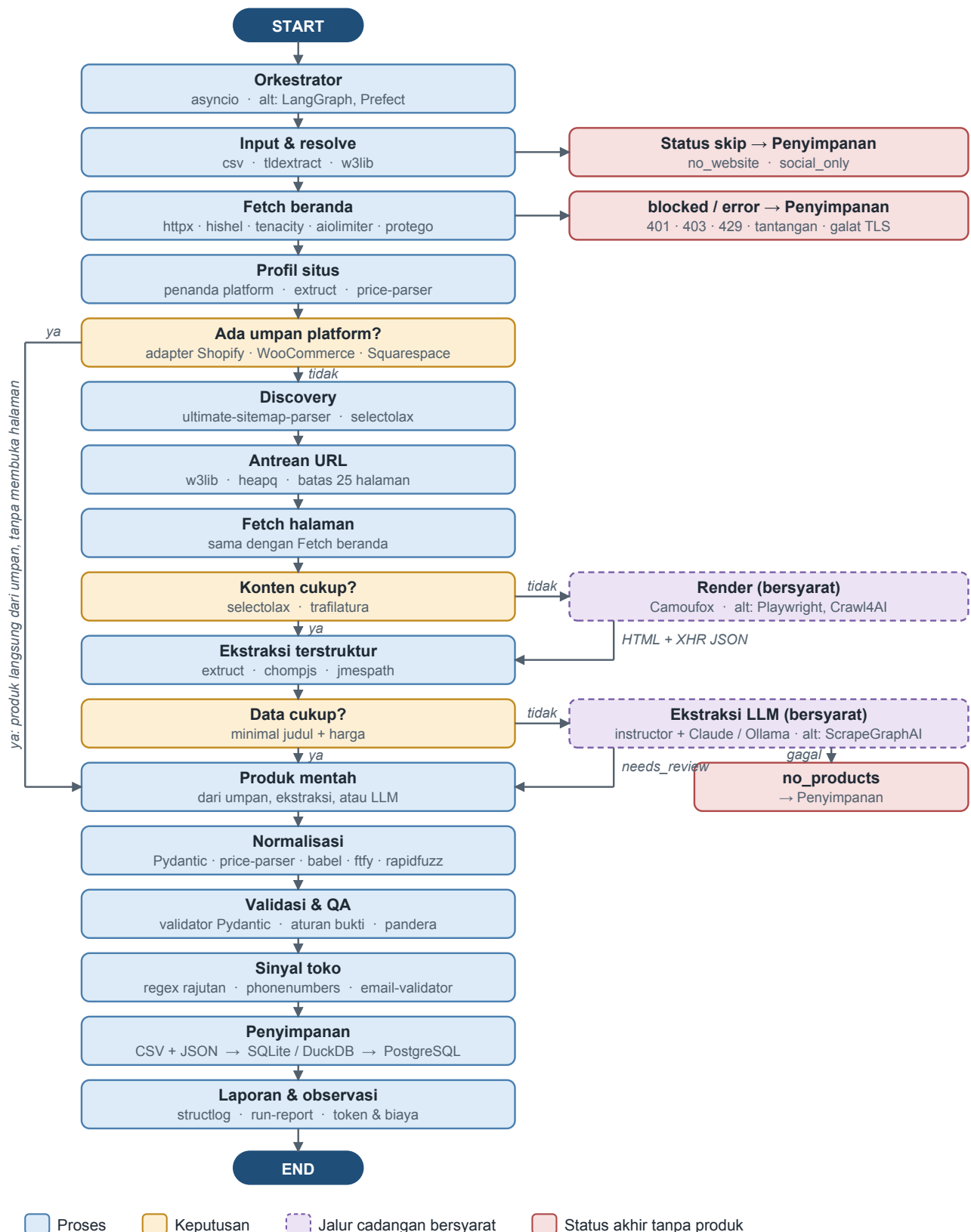
1. Ringkasan

Dokumen ini menjabarkan alur kerja teknis `scrapebot` dari awal sampai akhir: data apa yang masuk ke setiap tahap, apa tujuannya, pustaka apa yang dipakai, dan apa keluarannya. Prinsip dasarnya tetap sama dengan rancangan versi 2:

- **Cara termurah dan paling akurat dicoba lebih dahulu.** Umpan produk platform mendahului pembacaan HTML, pembacaan HTML mendahului peramban, dan peramban mendahului LLM.
- **Setiap baris masukan menghasilkan tepat satu baris keluaran**, lengkap dengan status dan asal datanya.
- **Tidak ada kontrol akses yang ditembus.** `robots.txt` dipatuhi, dan situs yang memblokir dicatat sebagai `blocked`.

Yang baru di dokumen ini adalah dua hal. Pertama, setiap tahap dirancang sebagai *plug-in* dengan antarmuka yang tetap, sehingga implementasinya dapat diganti lewat konfigurasi. Kedua, pilihan final setiap tahap ditentukan melalui benchmark pada sekumpulan situs yang jawabannya sudah diperiksa manual.

2. Diagram Alur Teknis



Gambar 1. Alur kerja teknis. Baris kedua di setiap kotak adalah pustaka yang direkomendasikan. Kotak bergaris putus-putus hanya dijalankan bila tahap sebelumnya tidak cukup.

3. Rincian Setiap Tahap

Setiap tahap dijelaskan dengan format yang sama: tujuan, pustaka yang direkomendasikan, alternatifnya, keluaran, dan catatan praktik terbaik.

3.0 Orkestrator

Tujuan	Menjalankan setiap toko melalui semua tahap, mencatat jejak tahap yang dilalui, dan dapat dilanjutkan bila proses terhenti.
Rekomendasi	<code>asyncio</code> dengan batas konkurensi per domain (<code>asyncio.Semaphore</code>). Ringan, tanpa dependensi tambahan, dan mudah diuji.
Alternatif	LangGraph : <code>StateGraph</code> dengan checkpoint dan visualisasi graf; paling berguna bila ada langkah yang diputuskan oleh LLM. Prefect : penjadwalan, pemantauan, dan coba ulang per tugas; cocok bila bot dijalankan berkala.
Keluaran	Jejak tahap per toko (<code>layers_tried</code>).
Praktik terbaik	Setiap tahap berupa fungsi dengan masukan dan keluaran Pydantic. Dengan begitu orkestrator dapat diganti tanpa mengubah isi tahap.

3.1 Input dan resolusi

Tujuan	Daftar domain yang bersih dan tanpa duplikat; setiap baris masukan tetap menghasilkan tepat satu baris keluaran.
Rekomendasi	<code>csv</code> (pustaka standar), <code>tlldextract</code> untuk domain terdaftar yang benar (termasuk akhiran seperti <code>.co.uk</code>), <code>w3lib</code> untuk normalisasi URL.
Alternatif	<code>pandas</code> , bila daftar masukan besar atau perlu banyak transformasi.
Keluaran	Daftar <code>Target</code> ; baris yang dilewati berstatus <code>no_website</code> atau <code>social_only</code> .

3.2 Fetch (beranda dan halaman)

Tujuan	Akses jaringan yang sopan, tersimpan di cache, dan tahan terhadap galat.
Rekomendasi	<code>httpx</code> (async, HTTP/2), <code>hishel</code> (cache HTTP untuk <code>httpx</code>), <code>tenacity</code> (coba ulang dengan jeda bertahap), <code>aiolimiter</code> (jeda per domain), <code>protego</code> (pengurai <code>robots.txt</code> dari ekosistem Scrapy yang mendukung wildcard).
Alternatif	requests + requests-cache : yang dipakai sekarang; cukup bila tidak perlu paralel. Scrapy : <code>robots</code> , <code>autothrottle</code> , <code>cache</code> , dan coba ulang sudah bawaan, tetapi berupa kerangka kerja penuh yang lebih sulit digabung dengan Camoufox dan alur bertahap.
Keluaran	<code>FetchResult</code> ; status <code>blocked</code> untuk 401, 403, 429, dan halaman tantangan; status <code>error</code> untuk galat jaringan atau TLS.
Praktik terbaik	Hanya respons yang berhasil disimpan di cache. Header <code>Accept-Language</code> tidak dikirim agar harga tidak dilokalkan.

3.3 Profil situs

Tujuan	Mengetahui platform dan mata uang sebelum memilih strategi pembacaan.
Rekomendasi	Aturan penanda platform sendiri (sudah ada), <code>extract</code> untuk <code>priceCurrency</code> , <code>price-parser</code> untuk mengurai teks harga beserta simbol mata uangnya.
Alternatif	Basis data sidik jari teknologi ala Wappalyzer.
Keluaran	Platform, kode mata uang, dan sumber mata uang tersebut.

3.4 Umpan produk platform

Tujuan	Mengambil data produk yang paling akurat dan gratis tanpa membuka halaman produk satu per satu.
Rekomendasi	Adapter sendiri untuk setiap platform: Shopify <code>/products.json</code> , WooCommerce Store API, Squarespace <code>?format=json</code> , dan Lightspeed (diverifikasi saat survei).
Alternatif	Tidak ada yang sepadan; ini jalur tercepat dan paling murah.
Keluaran	Produk mentah langsung, tanpa melewati tahap 3.5–3.10.
Praktik terbaik	Adapter yang cocok dengan platform terdeteksi dicoba lebih dahulu; bila platform tidak diketahui, semua adapter dicoba.

3.5 Discovery (penemuan halaman)

Tujuan	Menemukan halaman produk, kontak, “tentang kami”, dan grosir.
Rekomendasi	<code>ultimate-sitemap-parser</code> (indeks sitemap, berkas gzip, dan sitemap yang tercantum di <code>robots.txt</code>), <code>selectolax</code> (ekstraksi tautan yang cepat).
Alternatif	<code>Scrapy SitemapSpider</code> , fitur <i>deep crawl</i> <code>Crawl4AI</code> .
Keluaran	Kandidat URL.

3.6 Antrean URL

Tujuan	URL yang kanonik, tanpa duplikat, terurut menurut prioritas, dan jumlahnya dibatasi.
Rekomendasi	<code>w3lib.url.canonicalize_url</code> , <code>heapq</code> untuk prioritas, batas 25 halaman per toko.
Alternatif	Redis, bila nanti memakai banyak worker di beberapa mesin.
Keluaran	Antrean URL; halaman kontak, “tentang kami”, dan grosir di urutan depan.

3.7 Pemeriksaan konten

Tujuan	Hanya me-render halaman yang memang memerlukannya.
Rekomendasi	<code>selectolax</code> dan <code>trafilatura</code> : panjang teks utama, kerangka aplikasi kosong (mis. <code><div id="root"></div></code>), dan pesan <code><noscript></code> .
Keluaran	Keputusan render: ya atau tidak, per halaman.
Praktik terbaik	Ambang batasnya ditetapkan dari data survei, bukan ditebak.

3.8 Render peramban (bersyarat)

Tujuan	Membaca situs JavaScript dan menangkap JSON produk yang dimuat halaman.
Rekomendasi	Camoufox : Firefox dengan sidik jari realistis dan API Playwright; respons XHR ditangkap dengan <code>page.on("response")</code> .
Alternatif	Playwright-Chromium : lebih umum, tetapi lebih mudah dikenali sebagai bot. Crawl4AI : pengelolaan peramban siap pakai.
Keluaran	HTML hasil render serta JSON XHR beserta URL sumbernya.
Praktik terbaik	Maksimal 10 halaman per toko; pemeriksaan robots dan jeda sama dengan HTTP; lokal <code>en-US</code> ; halaman tantangan dicatat sebagai <code>blocked</code> , tidak ditembus.

3.9 Ekstraksi data terstruktur

Tujuan	Mendapatkan data produk yang tepat tanpa LLM.
Rekomendasi	<code>extract</code> (JSON-LD, Microdata, OpenGraph, RDFa), <code>chompjs</code> (mengurai objek JavaScript seperti <code>window.__INITIAL_STATE__</code> yang bukan JSON murni), <code>jmespath</code> (mengambil field dari JSON bertingkat).
Keluaran	Produk dengan sumber <code>structured_data</code> atau <code>browser_xhr</code> .

3.10 Ekstraksi LLM (bersyarat)

Tujuan	Membaca situs yang sama sekali tidak memiliki data terstruktur; setiap produk wajib menyertakan URL bukti.
Rekomendasi	instructor + Pydantic: satu API untuk Claude, OpenAI, dan Ollama; keluaran divalidasi dengan skema dan dicoba ulang otomatis bila tidak valid. Model awal Claude Haiku 4.5; Ollama sebagai opsi gratis. HTML diringkas lebih dahulu dengan <code>trafilatura</code> atau <code>markdownify</code> .
Alternatif	ScrapeGraphAI <code>SmartScraperGraph</code> : sumber terbuka, dapat menerima HTML milik bot sebagai <code>source</code> , tetapi menarik <code>LangChain</code> . Crawl4AI <code>LLMExtractionStrategy</code> (melalui LiteLLM). SDK Anthropic langsung : paling sedikit dependensi.
Keluaran	Produk dengan <code>needs_review = True</code> ; catatan versi prompt, model, jumlah token, dan respons mentah.
Praktik terbaik	Hanya berjalan untuk toko yang masih nol produk. Produk yang URL buktinya bukan halaman yang dikirim akan dibuang.

3.11 Normalisasi

Tujuan	Menyeragamkan produk dari semua sumber menjadi satu bentuk data.
Rekomendasi	Pydantic (model <code>Product</code>), <code>price-parser</code> (angka dan mata uang dari teks), <code>babel</code> (kode mata uang ISO 4217), <code>ftfy</code> (memperbaiki teks rusak akibat salah encoding), <code>rapidfuzz</code> (menggabungkan judul yang hampir sama).
Keluaran	Produk ternormalisasi.
Praktik terbaik	Harga WooCommerce dalam satuan sen dikonversi di sini, bukan di adapter, agar aturan konversi berada di satu tempat.

3.12 Validasi dan QA

Tujuan	Menolak data yang salah dan menandai data yang meragukan.
Rekomendasi	Validator Pydantic (judul tidak kosong, harga positif, mata uang ISO), aturan bukti untuk hasil LLM, <code>pandera</code> untuk memeriksa tabel akhir (kolom, tipe, rentang nilai).
Keluaran	Produk yang valid serta jumlah produk yang gugur beserta alasannya.
Praktik terbaik	Data salah yang tersembunyi lebih buruk daripada celah yang terlihat. Semua produk yang gugur dihitung di laporan.

3.13 Sinyal toko

Tujuan	Menghasilkan bukti yang akan dinilai manusia: rajutan, rentang harga, dan kontak.
Rekomendasi	Regex rajutan (sudah ada), <code>phonenumbers</code> (format telepon internasional, memperbaiki masalah v1 nomor 5), <code>email-validator</code> .
Alternatif	Klasifikasi rajutan dengan LLM untuk judul yang ambigu.
Keluaran	<code>knit_count</code> , <code>knit_share</code> , rentang harga dalam satu mata uang, kontak, halaman grosir, dan penanda jaringan ritel.

3.14 Penyimpanan

Tujuan	Hasil yang tahan lama dan dapat dinilai ulang tanpa scraping ulang.
Rekomendasi	CSV hasil + JSON mentah per domain (seperti sekarang).
Alternatif	SQLite (<code>sqlite-utils</code>) atau DuckDB (kueri analitik langsung di atas CSV, JSON, atau Parquet tanpa server). PostgreSQL + SQLAlchemy bila bot berjalan berkala dan datanya dibaca banyak orang atau layanan.
Praktik terbaik	Penyimpanan berupa penulis tambahan atas rekaman yang sama, sehingga beralih ke basis data tidak mengubah alur.

3.15 Laporan dan observasi

Tujuan	Mengetahui cakupan, galat, dan biaya setiap run.
Rekomendasi	<code>structlog</code> (log terstruktur JSON), <code>run-report.md</code> , pencatatan token dan biaya LLM.
Alternatif	LangSmith (bila memakai LangGraph), dasbor Prefect.
Keluaran	Tabel cakupan per sumber, daftar <code>no_products</code> dan <code>needs_review</code> , produk yang gugur validasi, tahap yang dilewati, total token.

3.16 Pengujian (lintas tahap)

Tujuan	Semua pengujian berjalan tanpa jaringan dan hasilnya selalu sama.
Rekomendasi	<code>pytest</code> , <code>pytest-recording</code> (merekam respons HTTP nyata sebagai <i>cassette</i>), <code>respx</code> (tiruan <code>httpx</code>), <code>fixture</code> respons LLM yang direkam.
Alternatif	<code>responses</code> untuk <code>requests</code> , <code>vcrpy</code> langsung.

4. Arsitektur Plug-in agar Dinamis

Pola yang dipakai adalah *ports and adapters*. Setiap tahap mendefinisikan satu antarmuka (port), dan setiap pustaka dibungkus sebagai satu implementasi (adapter). Adapter yang aktif dipilih melalui berkas konfigurasi.

4.1 Berkas konfigurasi

```
# config.yaml
orchestrator: asyncio           # asyncio | langgraph
fetch: httpx                    # httpx | requests
render: camoufox                 # camoufox | playwright | none
llm:
  backend: instructor           # instructor | scrapegraph | crawl4ai | none
  model: claude-haiku-4-5       # atau ollama/qwen2.5:7b
storage: [csv, json]            # tambahkan sqlite | postgres bila perlu
```

4.2 Antarmuka tahap

```
class LLMExtractor(Protocol):
    def extract(self, pages: list[Page],
                schema: type[BaseModel]) -> ExtractResult: ...

# InstructorExtractor, ScrapeGraphExtractor, Crawl4AIExtractor
# memenuhi antarmuka yang sama dan dapat saling menggantikan.
```

Port (antarmuka)	Adapter yang disiapkan
Fetcher	httpx, requests
Renderer	Camoufox, Playwright, tiruan untuk pengujian
FeedAdapter	Shopify, WooCommerce, Squarespace, Lightspeed
StructuredExtractor	extract, app state (chompjs), XHR JSON
LLMExtractor	instructor (Claude / OpenAI / Ollama), ScrapeGraphAI, Crawl4AI
Storage	CSV, JSON, SQLite, PostgreSQL

4.3 Manfaat

- Mengganti ScrapeGraphAI dengan instructor, atau asyncio dengan LangGraph, cukup dengan mengubah satu baris konfigurasi.
- Karena setiap tahap adalah fungsi dengan masukan dan keluaran Pydantic, diagram pada Gambar 1 dapat dipetakan langsung ke `StateGraph` LangGraph: tahap menjadi *node*, dan kotak keputusan menjadi *conditional edge*.
- Pengujian memakai adapter tiruan, sehingga tetap berjalan tanpa jaringan.
- Benchmark (bagian 5) dapat menjalankan beberapa adapter pada data yang sama.

5. Memilih yang Final melalui Benchmark

Pilihan pustaka diputuskan dengan pengukuran, bukan opini. Benchmark dijalankan bersamaan dengan Tahap 1 (survei) pada rencana kerja, sehingga tidak menambah fase baru.

5.1 Langkah

1. **Siapkan gold set:** sekitar 20 situs non-Shopify dari daftar calon mitra. Produk dan harganya diperiksa manual sebagai kunci jawaban.
2. **Rekam halamannya** agar setiap kandidat membaca masukan yang persis sama.
3. **Jalankan setiap kandidat** pada gold set, misalnya instructor + Claude, instructor + Ollama, ScrapeGraphAI, dan Crawl4AI untuk tahap LLM.
4. **Bandingkan dengan metrik** pada tabel di bawah.
5. **Tetapkan pemenang** per tahap sebagai bawaan di `config.yaml`.

5.2 Metrik

Metrik	Cara mengukur
Ketepatan (presisi)	Dari produk yang ditemukan, berapa persen yang benar
Kelengkapan (recall)	Dari produk di kunci jawaban, berapa persen yang ditemukan
Ketepatan harga	Persentase harga dan mata uang yang sama persis dengan kunci jawaban
Biaya	Token LLM dan waktu peramban per toko
Waktu	Durasi rata-rata per toko
Stabilitas	Apakah dua kali perjalanan pada masukan yang sama memberi hasil sama
Perawatan	Jumlah dependensi, ukuran instalasi, dan keaktifan proyek

Aturan keputusan yang disarankan: pilih kandidat termurah yang ketepatan dan kelengkapannya berada dalam 5 poin persentase dari kandidat terbaik. Bila selisihnya lebih besar, pilih yang terbaik dan catat biayanya.

6. Praktik Terbaik

Praktik	Penerapan di scrapebot
Termurah dan paling akurat dahulu	Umpan, lalu data terstruktur, lalu peramban, lalu LLM
Satu tanggung jawab per tahap	Setiap tahap punya satu tujuan dan satu antarmuka
Data bertipe di setiap batas	Masukan dan keluaran tahap berupa model Pydantic
Idempoten dan dapat dilanjutkan	Cache respons yang berhasil dan jejak tahap per toko
Sopan terhadap situs	<code>robots.txt</code> , jeda per domain, tanpa login, tanpa penembusan antibot
Asal data di setiap baris	<code>source_used</code> , <code>layers_tried</code> , URL bukti, <code>needs_review</code>
Gagal secara terlihat	Setiap toko mendapat status; produk yang gugur dihitung
Pengujian tanpa jaringan	Respons HTTP dan LLM direkam sebagai fixture
Biaya terpantau	Token dan waktu peramban dicatat di setiap run
Dependensi terkunci dan terpisah	Versi dikunci; peramban dan LLM sebagai dependensi opsional
Rahasia di luar kode	Kunci API di <code>.env</code> , tidak masuk repositori

7. Rekomendasi Awal dan Langkah Berikutnya

Tahap	Bawaan awal	Kandidat benchmark
Orkestrator	asyncio	LangGraph (bila perlu checkpoint atau visualisasi)
Fetch	httpx + hishel + tenacity	requests (yang sekarang)
Render	Camoufox	Playwright-Chromium
Ekstraksi terstruktur	extract + chompjs	—
Ekstraksi LLM	instructor + Claude Haiku 4.5	instructor + Ollama, ScrapeGraphAI, Crawl4AI
Penyimpanan	CSV + JSON	SQLite / DuckDB, PostgreSQL bila berkala

Langkah berikutnya:

1. Selesaikan Tahap 0 (perbaikan v1) dan susun antarmuka plug-in untuk setiap tahap.
2. Siapkan gold set 20 situs dan rekam halamannya.
3. Jalankan survei dan benchmark sekaligus, lalu tinjau hasilnya.
4. Tetapkan bawaan final di `config.yaml` berdasarkan aturan keputusan pada bagian 5.

Kesimpulan. Alur kerja ini memakai pustaka khusus yang kecil untuk setiap tahap, membungkusnya sebagai plug-in, dan memilih yang terbaik berdasarkan data. Hasilnya dinamis, karena pustaka mana pun dapat dicoba dan diganti, dan tetap hemat, karena peramban dan LLM hanya dipakai bila tahap yang lebih murah tidak cukup.